

Computer Networks

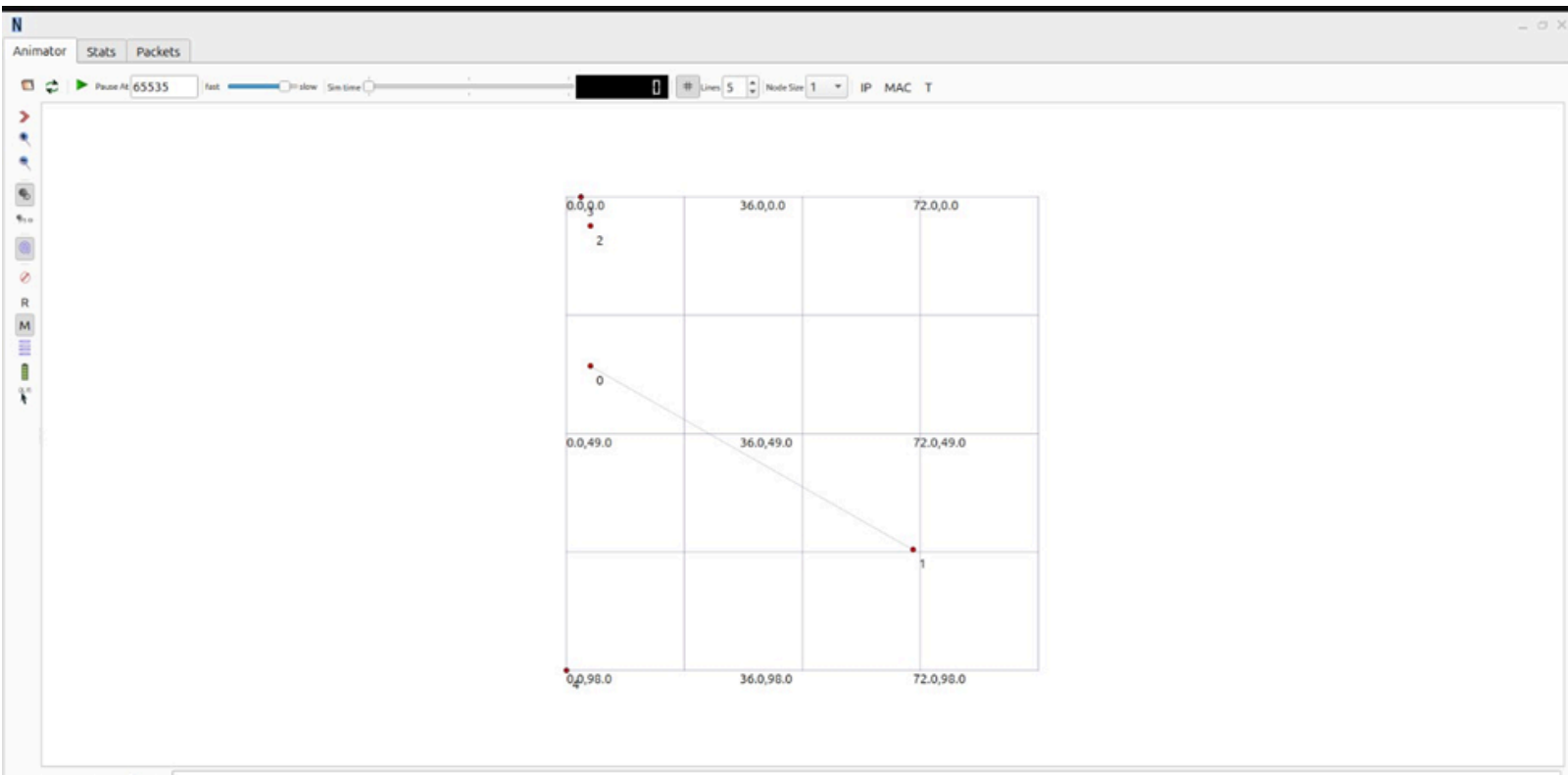
LAB#07

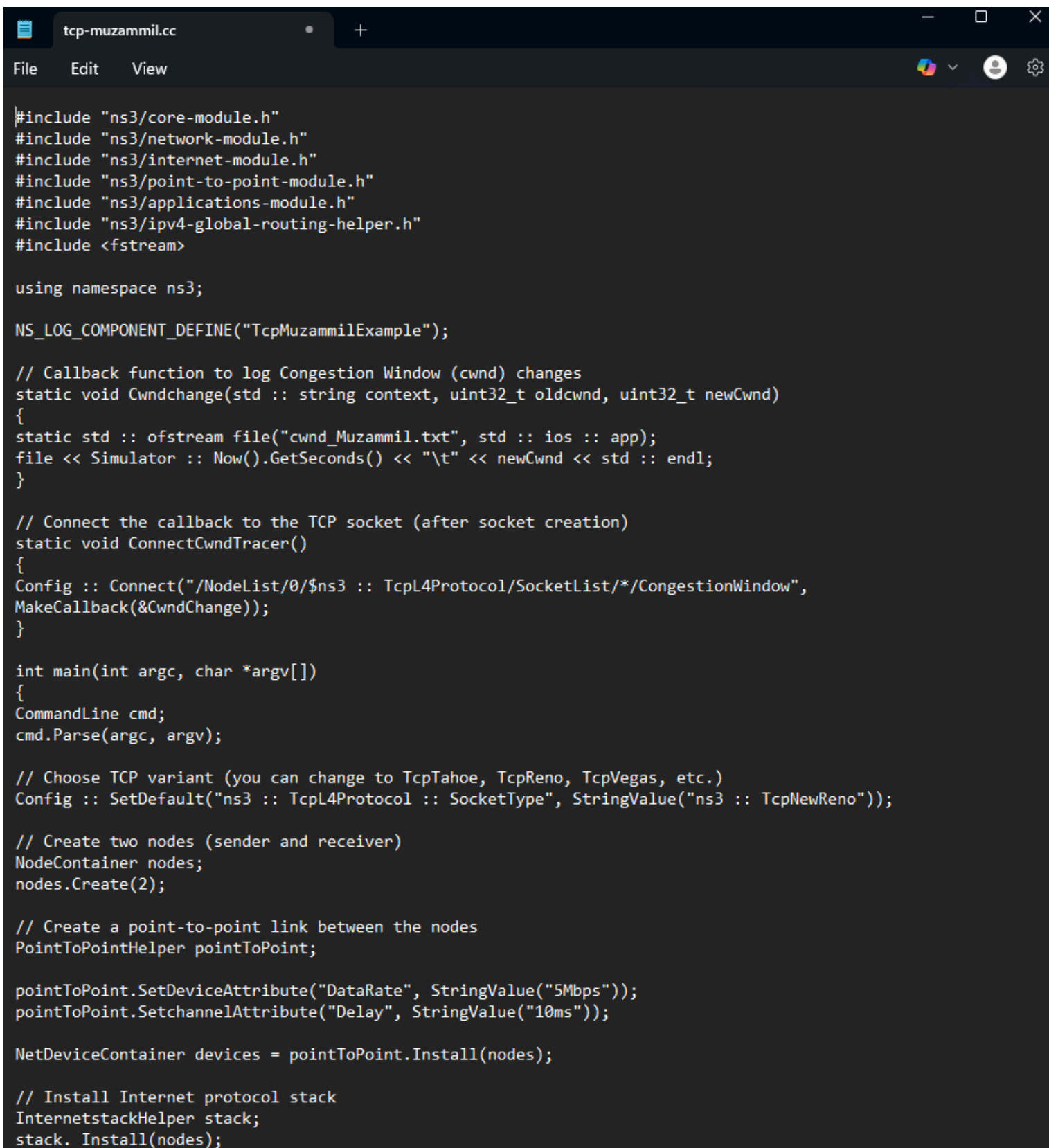


23K-2001

BCS-5J

Q1:





```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/ipv4-global-routing-helper.h"
#include <fstream>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("TcpMuzammilExample");

// Callback function to log Congestion Window (cwnd) changes
static void Cwndchange(std :: string context, uint32_t oldcwnd, uint32_t newCwnd)
{
    static std :: ofstream file("cwnd_Muzammil.txt", std :: ios :: app);
    file << Simulator :: Now().GetSeconds() << "\t" << newCwnd << std :: endl;
}

// Connect the callback to the TCP socket (after socket creation)
static void ConnectCwndTracer()
{
    Config :: Connect("/NodeList/0/$ns3 :: TcpL4Protocol/SocketList/*/CongestionWindow",
    MakeCallback(&Cwndchange));
}

int main(int argc, char *argv[])
{
    CommandLine cmd;
    cmd.Parse(argc, argv);

    // Choose TCP variant (you can change to TcpTahoe, TcpReno, TcpVegas, etc.)
    Config :: SetDefault("ns3 :: TcpL4Protocol :: SocketType", StringValue("ns3 :: TcpNewReno"));

    // Create two nodes (sender and receiver)
    NodeContainer nodes;
    nodes.Create(2);

    // Create a point-to-point link between the nodes
    PointToPointHelper pointToPoint;

    pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
    pointToPoint.SetchannelAttribute("Delay", StringValue("10ms"));

    NetDeviceContainer devices = pointToPoint.Install(nodes);

    // Install Internet protocol stack
    InternetstackHelper stack;
    stack.Install(nodes);
```

```

// Assign IP addresses
Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign(devices);

// Set up a packet sink (receiver)
uint16_t port = 8080;
Address sinkAddress(InetSocketAddress(interfaces.GetAddress(1), port));

PacketSinkHelper packetSinkHelper("ns3 :: TcpSocketFactory",
InetSocketAddress(Ipv4Address :: GetAny(), port));
ApplicationContainer sinkApp = packetSinkHelper. Install(nodes.Get(1));
sinkApp.Start(Seconds(0.0));
sinkApp.Stop(Seconds(20.0));

// Set up a BulkSend application (sender)
BulkSendHelper bulkSender("ns3 :: TcpSocketFactory", sinkAddress);
bulkSender.SetAttribute("MaxBytes", UintegerValue(0)); // unlimited data
ApplicationContainer sourceApp = bulkSender. Install(nodes.Get(0));
sourceApp.Start(Seconds(1.0));
sourceApp.Stop(Seconds(20.0));

// Schedule cwnd tracer to start after the sender begins
Simulator :: Schedule(Seconds(1.1), &ConnectCwndTracer);

// Run simulation
Simulator :: Stop(Seconds(20.0));
Simulator: : Run();
Simulator :: Destroy();

NS_LOG_UNCOND("Check cwnd_Muzammil.txt for results.");
return 0;
}

```

